1. CFG Tree vs Dependency Parsing

```
PS C:\Users\K.RAMESH\OneDrive\Desktop\CO4_AT1(Data Interpretation)> & C:\Users\K.RAMESH\AppData\Local\Programs\Python\Python314\python.exe "c:/U
sers/K.RAMESH/OneDrive/Desktop/CO4_AT1(Data Interpretation)/11.py"
1. CFG REPRESENTATION
S
|
+-- NP
|   +-- The
|   +-- student
|
+-- VP
    +-- solved
    +-- NP
        +-- the
        +-- problem

2. DEPENDENCY REPRESENTATION
solved
 |
+-- student (subject)
|   +-- The (determiner)
|
+-- problem (object)
    +-- the (determiner)

3. WORD RELATIONSHIPS
student -> subject of -> solved
problem -> object of  -> solved
```

2. Top-Down Parsing vs Earley Parsing

```
PS C:\Users\K.RAMESH\OneDrive\Desktop\CO4_AT1(Data Interpretation)> & C:\Users\K.RAMESH\AppData\Local\Programs\Python\Python314\python.exe "c:/U
sers/K.RAMESH/OneDrive/Desktop/CO4_AT1(Data Interpretation)/12.py"

2. EARLEY PARSING
S -> VP .
VP -> Verb NP .
PP -> to NP .
PP -> with NP .

Earley parser keeps partial states.
It can handle ambiguous and incomplete input.

3. PARTIAL INPUT
Input: Book a flight to
Current state:
PP -> to . NP
Waiting for destination...

4. COMPARISON
Top-Down:
- Uses prediction
- May require backtracking
- Less suitable for incomplete input

Earley:
- Maintains partial parsing states
- Handles ambiguity
```

3. CFG vs PCFG vs Neural Parsing

```
PS C:\Users\K.RAMESH\OneDrive\Desktop\CO4_AT1(Data Interpretation)> & C:\Users\K.RAMESH\AppData\Local\Programs\Python\Python314\python.exe "c:/U
sers/K.RAMESH/OneDrive/Desktop/CO4_AT1(Data Interpretation)/13.py"
Parse 1:
She [saw the man] [with a telescope]
Meaning: She used a telescope.

Parse 2:
She [saw [the man with a telescope]]
Meaning: The man had a telescope.

CFG RESULT:
CFG can generate both possible parses.

2. PCFG
Parse 1 probability: 0.7
Parse 2 probability: 0.3
PCFG selects Parse 1.

PCFG uses probabilities to rank parses.

3. NEURAL PARSING
Input words:
She
saw
the
man
with
```

4. Feature Structures vs Sub categorization Frames

```
PS C:\Users\K.RAMESH\OneDrive\Desktop\CO4_AT1(Data Interpretation)> & C:\Users\K.RAMESH\AppData\Local\Programs\Python\Python314\python.exe "c:/U
sers/K.RAMESH/OneDrive/Desktop/CO4_AT1(Data Interpretation)/14.py"
1. FEATURE STRUCTURE
Subject: {'WORD': 'student', 'PERSON': 3, 'NUMBER': 'singular'}
Verb   : {'WORD': 'writes', 'PERSON': 3, 'NUMBER': 'singular'}
Agreement: CORRECT

Incorrect Example
Subject: {'WORD': 'student', 'PERSON': 3, 'NUMBER': 'singular'}
Verb   : {'WORD': 'write', 'PERSON': 3, 'NUMBER': 'plural'}
Agreement: ERROR

2. SUBCATEGORIZATION FRAMES
eat -> NP
give -> NP + NP
sleep -> No Object
recommend -> Action

Examples:
eat -> The patient ate medicine.
give -> Doctor gave medicine to patient.
sleep -> The patient slept.
recommend -> Doctor recommends treatment.

3. CONCLUSION
Feature structures enforce agreement.
Subcategorization frames enforce verb argument structure.
```

5. Transition-Based vs Graph-Based Dependency Parsing

```
PS C:\Users\K.RAMESH\OneDrive\Desktop\CO4_AT1(Data Interpretation)> & C:\Users\K.RAMESH\AppData\Local\Programs\Python\Python314\python.exe "c:/U
sers/K.RAMESH/OneDrive/Desktop/CO4_AT1(Data Interpretation)/15.py"
SHIFT
Stack : ['The', 'student', 'solved']
Buffer: ['problems']

RIGHT-ARC
student -> subject -> solved

SHIFT
Stack : ['The', 'student', 'solved', 'problems']
Buffer: []

RIGHT-ARC
problems -> object -> solved

Transition-based parser makes local decisions step by step.

2. GRAPH-BASED PARSING
Possible dependency relationships:
student -> subject -> solved
problems -> object -> solved
The -> determiner -> student

Graph-based parser evaluates possible edges
and selects the best complete dependency tree.
```